

PROGRAMMING ASSIGNMENT 1: DECISION TREES

Due: Friday 10/02/2026 @ 11:59pm EST

The purpose of programming assignments is to implement model(s) we discuss in lecture and to then perform experiments on them. We are primarily using the Python programming language in this version of the assignments. Your task this assignment is to implement the Decision Tree Algorithm from lecture as well as the Bagging and Boosting Algorithms. Finally, you will assemble an ensemble of Decision Trees to make a Random Forest.

Task 1: Required modules

In order to build/run this assignment you will need to install some packages if you are not using the SCC:

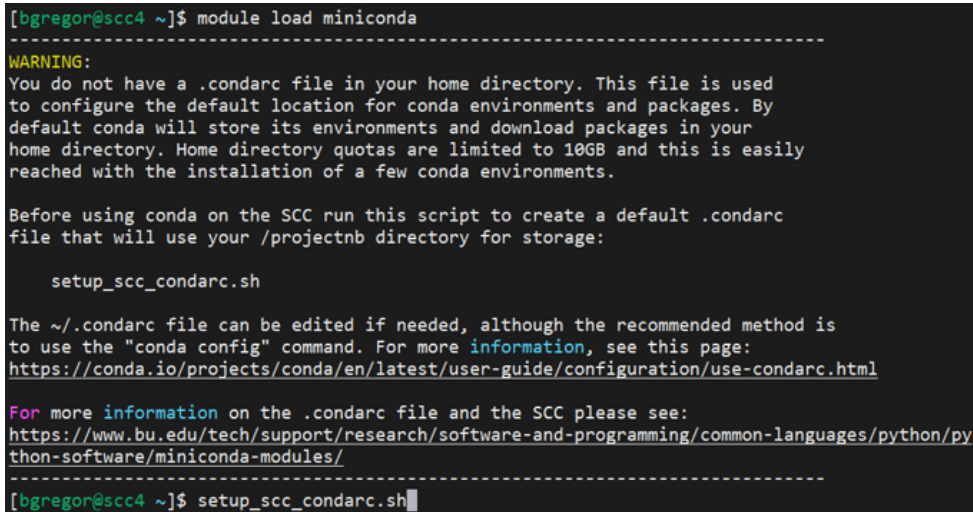
- If you are developing locally I have provided a `requirements.txt` file which you can use to install the required dependencies with the command:

```
python3 -m pip install -r requirements.txt
```

- If you are using the SCC then you will need to load the correct modules after logging in. You can do so with the following terminal commands:

```
module load miniconda
```

This *may* cause a bunch of stuff to be printed out similar to the following image:



```
[bgregor@scc4 ~]$ module load miniconda
-----
WARNING:
You do not have a .condarc file in your home directory. This file is used
to configure the default location for conda environments and packages. By
default conda will store its environments and download packages in your
home directory. Home directory quotas are limited to 10GB and this is easily
reached with the installation of a few conda environments.

Before using conda on the SCC run this script to create a default .condarc
file that will use your /projectnb directory for storage:

  setup_scc_condarc.sh

The ~/.condarc file can be edited if needed, although the recommended method is
to use the "conda config" command. For more information, see this page:
https://conda.io/projects/conda/en/latest/user-guide/configuration/use-condarc.html

For more information on the .condarc file and the SCC please see:
https://www.bu.edu/tech/support/research/software-and-programming/common-languages/python/python-software/miniconda-modules/
-----
[bgregor@scc4 ~]$ setup_scc_condarc.sh
```

If you see this message or something like that its ok! This just means you've never setup miniconda on the SCC before! This message is telling you that you need to run the following script from your SCC terminal:

```
setup_scc_condarc.sh
```

This script will perform some setup on your behalf. Something special to note about the SCC is that your home directory only has a 10GB quota, and its **very** easy to exhaust that storage if you let miniconda write to your home directory! If you run this script, it will configure miniconda to **not** use your home directory (which is good!).

If you **didn't** see this message, you might want to consider updating your `~/.condarc` file to point to the `cs542` directory on the SCC. If you didn't see this message then it is almost certain your `~/.condarc` is telling `miniconda` to write to some **other** class's/project's directory and you want to switch it for this class!

Anyways, once `miniconda` is setup, you can load and activate the following modules to get into a nice python environment:

```
module load miniconda # you already did this
module load academic-ml
conda activate fall-2026-pyt
```

The environment `fall-2026-pyt` is for the fall semester of year 2026 specialized for `pytorch`. There are also `tensorflow` and `jax` versions but if we need a neural network library we'll be using `pytorch`. This environment will have everything we need for this class already installed!

Task 2. The Starter Code

Included with this pdf is a directory called `src`. Inside this directory is the programming assignment starter code, tests, and data. There is also a sample `train_dt.py` script which should give you an idea of how to use this api to train some models.

I have implemented quite a bit for you. I have implemented two quality functions: information gain and gain ratio which you can find in the `src.dt.quality` package. If you want to add more quality functions you're welcome to but for this assignment we will only be using information gain and gain ratio. In the `src.dt.data` package you will find some code for loading data stored in the `c4.5` format as well as some datatypes useful for describing the features in such a dataset. There are also four datasets of various difficulties. I have included one "toy" dataset called the `play_outside` dataset (this is the dataset we've used in lecture). There are also `voting`, `spam`, and `volcanoes` datasets. Each of the "real" datasets has a `.info` file in their corresponding directory which describes the dataset. Feel free to read them!

The meat of this assignment is in the `src.dt.clf` module (in the `src/dt/clf.py` file). This file contains a partial implementation of a decision tree classifier as well as a random forest classifier. This assignment is all about finishing these implementations, improving them, and then applying them to the three "real" datasets.

I have also included some starter unit tests in the `tests` directory. These unit tests are not exhaustive so you will need to add more!

Task 3: Interior Nodes (ugrads: required & 30 points, grads: required & 20 points)

In the file `src/dt/clf.py` you will find several datatypes. The one's you are interested in for this task are the `InteriorNode` and `DecisionTreeClassifier` types. I have created a `Node` datatype to represent an arbitrary node in the tree and then created two classes: `LeafNode` and `InteriorNode` to represent the two kinds of nodes that are possible. I have already completed the `LeafNode` type and you need to finish the implementation of the `InteriorNode` type:

- method `_eval_feature`. Remember, when creating an interior node, your node needs to greedily consider which feature (of the features that node is allowed to use) has the "best" quality. That is the purpose of this method: determine which feature is the "best." An `InteriorNode` has a quality function field and I have designed the `_pick_best_feature` method to always maximize quality. Your method needs to, given a dataset and a feature index, determine the quality of this feature as well as the values this feature will split the data on (e.g. which values will determine the children). Your method needs to work for both continuous and discrete features and an arbitrary, already specified quality function. Reminder that if a feature is continuous, you need to argmax over the different *versions* of that continuous feature. Each version is indexed by a potential threshold that continuous feature could be split on. I have split the functionality of calculating those thresholds into another method which you should call if needed!
- method `_get_continuous_feature_thresholds`. If a feature is continuous, an interior node needs to consider all potential thresholds that could be used to split that continuous feature. Remember the algorithm from lecture to calculate the potential thresholds: you need to implement it here!
- method `predict`. Given a single sample, determine which child node that sample corresponds to (e.g. "execute" the interior node on the sample to choose and return a child node).
- method `get_child_datasets`. This method will take a dataset and "execute" the interior node on each sample: samples will flow along branches of this interior node and be collected into

“child datasets” which are then returned. The datasets returned need to be in the same order as the split values for the feature chosen inside this `InteriorNode` (there is a field for this called `feature.split_values` which, if you set when choosing the “best” feature, you can use here!).

I really recommend that you test your code! I have provided a starter unit test file that will test on the `play_outside` dataset with information gain. You should get the same results as we did in lecture!

Task 4: Decision Tree (ugrads: required & 20 points, grads: required & 10 points)

In the file `src/dt/clf.py` you will find the type `DecisionTreeClassifier`. This type acts as a wrapper around a tree structure contained within the `Node` types. I have implemented some of this for you but you need to complete the following method called `_build`. This method builds the tree in whatever order you want (BFS or DFS) using whatever flavor of implementation you want (recursive or iterative). This method is where you will implement the decision tree learning algorithm: your method needs to decide, given a dataset, what kind of node to make for that dataset (e.g. a `LeafNode` or an `InteriorNode`). If an `InteriorNode` is built then your code needs to construct the children of that node by generating the “child” datasets and turning them into nodes. This process repeats until the tree is fully built. This method returns a `Node`. When the `fit` method calls this method, the returned `Node` is the root of the entire tree.

Task 5: Experiments (ugrads: required & 50 points, grads: required & 25 points)

Write a script (probably similar to `train_dt.py`) that will perform 5-fold stratified cross validation using a repeatable random seed of 12345 on whatever dataset you choose. Include in your report answers to the following questions under a section called **Decision Tree Experiments**:

- What is the average 5-fold stratified cross validation classification accuracy of a decision tree restricted to a max depth of 1 on each of the three “real” datasets using information gain? What is the variance?
- For the `volcanoes` and `spam` datasets, plot the average 5-fold stratified cross validation classification accuracy (including error bars) as the max depth of the tree is increased using information gain. On the x-axis choose depth values to test so that there are at least 5 evenly-spaced points. Does the accuracy improve smoothly as the depth of the tree increases? Can you explain the pattern in the graph?
- Pick 3 different max depth values. How do the average 5-fold stratified cross validation classification accuracies change if you change the quality function from information gain to gain ratio? How about the variances?
- Rather than perform 5-fold stratified cross validation, train a decision tree using information gain on the full dataset with a max depth of 1 and 2. Are the results comparable to the 5-fold stratified cross validation classification accuracies?

Task 6: Pruning (ugrads: bonus & 25 points, grads: required & 20 points)

Now modify your `fit` method in the `DecisionTreeClassifier` type. There is an extra argument in the signature of this method called `mcc_prune` and takes a boolean value. If this value is set to `True` your decision tree should build the decision tree like normal and then prune the tree afterwards using the **minimum cost complexity pruning** algorithm from lecture using the pessimistic error estimate (with hyperparameter $\alpha > 0$):

$$e' := e(D_{train}) + \alpha |leaves(T)|$$

where $e(\cdot)$ is the number of misclassifications on the training data. As a reminder, this algorithm builds a sequence of trees S where the sequence initially contains the fully built original tree. The algorithm then grows this sequence by adding a new tree calculated using the following steps:

1. Pick the most recently added tree to the sequence S denoted T .
2. Generate a new tree T' by replacing an interior node n^* with a leaf node where

$$n^* = \arg \min_{n \in InteriorNodes(T)} \frac{e'(prune(T, n)) - e'(T)}{|leaves(T)| - |leaves(prune(T, n))|}$$

and add T' to the end of the sequence S .

3. Repeat steps 1 and 2 until no new trees can be added to the sequence.
4. Choose the tree $T^* = \arg \min_{T \in S} e'(T)$

Task 7: Pruning Experiments (ugrads: bonus & 25 points, grads: required & 25 points)

Write a script (probably similar to `train_dt.py`) that will perform 5-fold stratified cross validation using a repeatable random seed of 12345 on whatever dataset you choose. Include in your report answers to the following questions under a section called **Pruning Experiments**:

- What is the average 5-fold stratified cross validation classification accuracy of a decision tree with post-pruning enabled on each of the three “real” datasets using information gain? What is the variance?
- For the `volcanoes` and `spam` datasets, plot the average 5-fold stratified cross validation classification accuracy (including error bars) as the hyperparameter α of the tree is increased using information gain. On the x-axis choose α values to test so that there are at least 5 evenly-spaced points. Does the accuracy improve smoothly as α increases? Can you explain the pattern in the graph? How do these compare against your results using pre-pruning (e.g. max depth)?
- Pick 3 different α values. How do the 5-fold stratified cross validation classification accuracies change if you change the quality function from information gain to gain ratio?
- Rather than perform 5-fold stratified cross validation, train a decision tree using information gain on the full dataset with α set to 0.5 and 1. Are the results comparable to the 5-fold stratified cross validation classification accuracies?

Task 8: Random Forest Bagging (ugrads: no credit, grads: bonus & 25 points)

In the file `src/dt/clf.py` you will find the type `RandomForestClassifier`. A random forest model is a flavor of a **bagging** ensemble. A ensemble is a model that wraps around a collection of simpler models which is used to train and predict (for instance having each simpler model vote and choosing the most popular label). Bagging is when you train a bunch of independent models (potentially in parallel) on subsets of the training data provided to the ensemble. In particular, a random forest model trains a collection of independent decision trees on subsets of the training data which it then uses to vote for the final prediction of the random forest.

I have already implemented some important functionality on your behalf: most notably is the functionality required to generate subsets of the training data. You will need to implement two methods to complete this implementation:

- method `fit`. This method is responsible for instantiating and training the forest of decision trees. For each decision tree you need to generate a random subset of the features to use as well as a bootstrap sampling of the original examples. I have provided two methods that perform these tasks, you will want to use them! Note that if pre and post pruning methods are requested each tree in the forest should be pruned.
- method `predict`. This method is responsible for asking each tree in the forest to predict the same samples and then decide which class is the most popular according to the forest.

Task 9: Random Forest Experiments (ugrads: no credit, grads: bonus & 25 points)

Write a script (probably similar to `train_dt.py`) that will perform 5-fold stratified cross validation using a repeatable random seed of 12345 on whatever dataset you choose. Include in your report answers to the following questions under a section called **Random Forest Experiments**:

- What is the average 5-fold stratified cross validation classification accuracy of a random forest with and without post-pruning enabled on each of the three “real” datasets using information gain and 100 trees? What is the variance?
- For the `volcanoes` and `spam` datasets, plot the average 5-fold stratified cross validation classification accuracy (including error bars) as the number of trees is increased using information gain. On the x-axis choose the number of trees to test so that there are at least 5 evenly-spaced points. Does the accuracy improve smoothly as the number of trees increases? Can you explain the pattern in the graph? How do these compare against your results using a single decision tree?
- Pick 3 different number of trees. How do the 5-fold stratified cross validation classification accuracies change if you change the quality function from information gain to gain ratio?
- Rather than perform 5-fold stratified cross validation, train a random forest using information gain on the full dataset with the number of trees set to 100 and 200. Are the results comparable to the 5-fold stratified cross validation classification accuracies?

Task 10: Submitting Your Assignment

Please turn in **only your source code** into gradescope as well as a written report. If you turn in any of the other files I **will not use them**. For the code autograders, I will put my own copies of the standard library and will be using **only** your `clf.py` file. Depending on how long the code autograders run for I may have to make multiple code autograders so be sure to submit to them all!